MiniBank (Part B)

- Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as "CUST101". Add a constructor and public getter methods.
- Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example "AC0001").
- Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0).
- Add a public method deposit(long amount) that adds amount to balance. Add a public method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods. Do NOT add a public setter for balance.
- In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance.

Source Code:

Customer.java:

```
package MiniBank;

public class Customer {
    private String name;
    private String email;
    private String mobile;
    private final String customerId;
    private static long customerCounter = 100;

    private static String generateCustomerId() {
        customerCounter++;
        return "CUST" + customerCounter;
    }

    public Customer(String name, String email, String mobile) {
        this.customerId = generateCustomerId();
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public String getCustomerId() { return customerId; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getMobile() { return mobile; }
}
```

Account.java:

```java
package MiniBank;

public class Account {
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;
    private static long accountCounter = 0;
    private static String generateAccountNumber() {
        accountCounter++;
        return String.format(format: "AC%04d", accountCounter);
    }

    // Constructor with owner name and opening balance
    public Account(String ownerName, long openingBalance) {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    // Deposit money
    public void deposit(long amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    // Withdraw money
    public boolean withdraw(long amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            return true;
        }
        return false;
    }

    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    public long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

MiniBank.java

```java
package MiniBank;
import java.util.Scanner;

...
// Record
record BankInfo(String name, String branch) { }

// Enum
enum MenuOption {OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, SHOW_DETAILS, EXIT }
```

```
You, 9 minutes ago | author (You)
public class MiniBank {
    // Find account using account number
    private static Account findAccount(
            Account[] accounts,
            int accountCount,
            String accountNumber) {

        for (int i = 0; i < accountCount; i++) {
            if (accounts[i].getAccountNumber().equals(accountNumber)) {
                return accounts[i];
            }
        }
        return null;
    }
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Bank information
        BankInfo bank = new BankInfo(name: "MiniBank", branch: "Anand Branch");
        System.out.println(x: "====================================");
        System.out.println(bank);
        System.out.println(x: "====================================");

        // Account array
        Account[] accounts = new Account[10];
        int accountCount = 0;
        boolean running = true;

        while (running) {
            System.out.println(x: "\n===== MINI BANK MENU =====");
            System.out.println(x: "1. Open Account");
            System.out.println(x: "2. Deposit");
            System.out.println(x: "3. Withdraw");
            System.out.println(x: "4. Transfer");
            System.out.println(x: "5. Show Account Details");
            System.out.println(x: "6. Exit");
            System.out.print(s: "Enter Choice : ");

            int choice = sc.nextInt();
            sc.nextLine();

            MenuOption option = switch (choice) {
                case 1 -> MenuOption.OPEN_ACCOUNT;
                case 2 -> MenuOption.DEPOSIT;
                case 3 -> MenuOption.WITHDRAW;
                case 4 -> MenuOption.TRANSFER;
                case 5 -> MenuOption.SHOW_DETAILS;
                case 6 -> MenuOption.EXIT;

                default -> null;
            };

            if (option == null) {
                System.out.println(x: "Invalid Choice!");
                continue;
            }
```

```
switch (option) {
    // 1. OPEN ACCOUNT
    case OPEN_ACCOUNT -> {
        if (accountCount >= accounts.length) {
            System.out.println(x: "Account limit reached!");
            break;
        }

        System.out.print(s: "Enter Owner Name : ");
        String ownerName = sc.nextLine();

        System.out.print(s: "Enter Opening Balance : ");
        long openingBalance = sc.nextLong();
        sc.nextLine();

        accounts[accountCount] = new Account(ownerName, openingBalance);
        System.out.println(x: "\nAccount Created Successfully!");
        System.out.println("Account Number : " + accounts[accountCount].getAccountNumber());
        accountCount++;
    }

    // 2. DEPOSIT
    case DEPOSIT -> {

        if (accountCount == 0) {
            System.out.println(x: "No accounts available!");
            break;
        }

        System.out.print(s: "Enter Account Number : ");
        String accountNumber = sc.nextLine();
        Account account = findAccount(accounts, accountCount, accountNumber);
        if (account == null) {
            System.out.println(x: "Account not found!");
            break;
        }

        System.out.print(s: "Enter Deposit Amount : ");
        long amount = sc.nextLong();
        sc.nextLine();
        account.deposit(amount);

        System.out.println(x: "Amount Deposited Successfully!");
        System.out.println("Current Balance : " + account.getBalance());
    }

    // 3. WITHDRAW
    case WITHDRAW -> {
        if (accountCount == 0) {
            System.out.println(x: "No accounts available!");
            break;
        }

        System.out.print(s: "Enter Account Number : ");
        String accountNumber = sc.nextLine();
        Account account = findAccount(accounts, accountCount, accountNumber);
        if (account == null) {
            System.out.println(x: "Account not found!");
            break;
        }
```

```
                System.out.print(s: "Enter Withdrawal Amount : ");
                long amount = sc.nextLong();
                sc.nextLine();

                boolean success = account.withdraw(amount);
                if (success) {
                    System.out.println(x: "Amount Withdrawn Successfully!");
                    System.out.println("Current Balance : "+ account.getBalance());
                } else {
                    System.out.println(x: "Withdrawal Failed! Insufficient Balance.");
                }
            }

            // 4. TRANSFER
            case TRANSFER -> {
                System.out.println(x: "Transfer - To be implemented in later lab.");
            }

            // 5. SHOW ACCOUNT DETAILS
            case SHOW_DETAILS -> {
                if (accountCount == 0) {
                    System.out.println(x: "No accounts available!");
                    break;
                }

                System.out.println(x: "\n===== ACCOUNT DETAILS =====");
                for (int i = 0; i < accountCount; i++) {
                    Account account = accounts[i];
                    System.out.println("Account Number : " + account.getAccountNumber());
                    System.out.println("Owner Name     : " + account.getOwnerName());
                    System.out.println("Balance        : " + account.getBalance());
                    System.out.println("Active         : " + account.isActive());
                    System.out.println(x: "----------------------------");
                }
                running = false;
            }

            // 6. EXIT
            case EXIT -> {
                System.out.println(x: "Thank You for using MiniBank.");
                running = false;
            }
        }
    }
}
```

MiniBank Output:

```
PS E:\SEM-3 CSPIT\OOP\Practicals\MiniBank> java .\MiniBank.java
====================================
BankInfo[name=MiniBank, branch=Anand Branch]
====================================

===== MINI BANK MENU =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Show Account Details
6. Exit
```

```
Enter Choice : 1
Enter Owner Name : Jigar Kotecha
Enter Operating Balance : 5000

Account Created Successfully!
Account Number : AC0001

===== MINI BANK MENU =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Show Account Details
6. Exit
Enter Choice : 2
Enter Account Number : AC0002
Account not found!

===== MINI BANK MENU =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Show Account Details
6. Exit
Enter Choice : 2
Enter Account Number : AC0001
Enter Deposit Amount : 5000
Amount Deposited Successfully!
Current Balance : 10000

===== MINI BANK MENU =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Show Account Details
6. Exit
Enter Choice : 5

===== ACCOUNT DETAILS =====
Account Number : AC0001
Owner Name     : Jigar Kotecha
Balance        : 10000
Active         : true
--------------------------------
```

GitHub Repository (MiniBank):

https://github.com/d26it118-png/OOP_PRACTICALS.git